上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

SJTU

Lecture 5 Serverless Computing Applications and Research Frontiers

马汝辉

上海交通大学

饮 水 思 源 • 爱 国 荣 校

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

目录



1 Opportunities

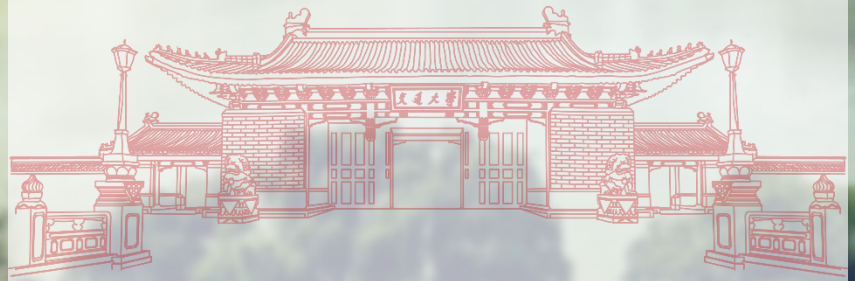
2 Applications

3 Challenges

4 Conclusion

01

Opportunities



No Deployment and Maintenance Complexity

- The very first and foremost opportunity that serverless computing offers is to relieve users from managing the infrastructure.
  - In Infrastructure-as-a-Service (IaaS), users still have to manage their **virtual resources**, i.e., installing and configuring related packages and libraries.
  - Certainly, Platform-as-a-Service (PaaS) providers such as Heroku have made the management slightly easier, although users still have to **configure the application** to match the PaaS requirements, which is not a trivial task.
  - Serverless computing takes a big step in this manner. Users only have to **register their functions** and then **receive credentials** to invoke the functions.

Affordable Scalability

- Another promise of cloud computing is the ability for customers to deploy the functionalities of their applications without worrying about the **scalability** of the execution infrastructure or platform.
- The scalability is a direct result of the **auto-scaling** nature of these services; i.e., per request the service invokes a copy of the requested function and there is virtually no bound for the number of concurrent requests. Each invocation is assigned to the most feasible and available resource for execution.

Affordable Scalability

- The affordability of serverless services is mainly due to the reduced costs of the providers. There are two main reasons for the reduced costs: (1) **resource multiplexing** and (2) **infrastructure heterogeneity.**
  - Resource multiplexing leads to higher utilization of available resources.
  - Infrastructure heterogeneity means that the providers also can use their older machines that are less attractive for other direct services to reduce their costs (mainly because the execution environment is a black box for the customer).

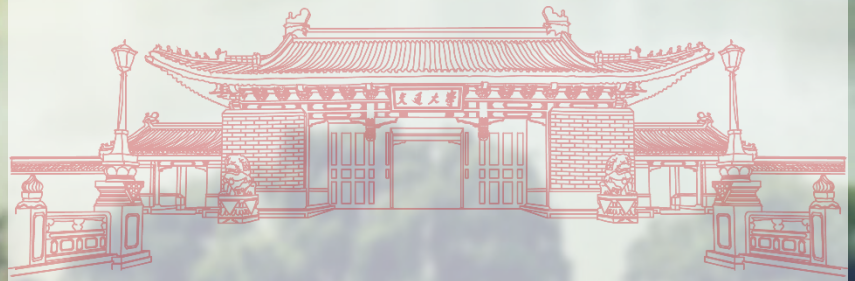
New Marketplaces

- With the growth in the popularity of serverless computing, new marketplaces for functions have emerged. In these types of markets, developers can sell their developed functions to others. Every generalized or domain-specific functionality can be bought or offered in those markets.
- The AWS Serverless Application Repository is an example of such a capability.



02

Applications



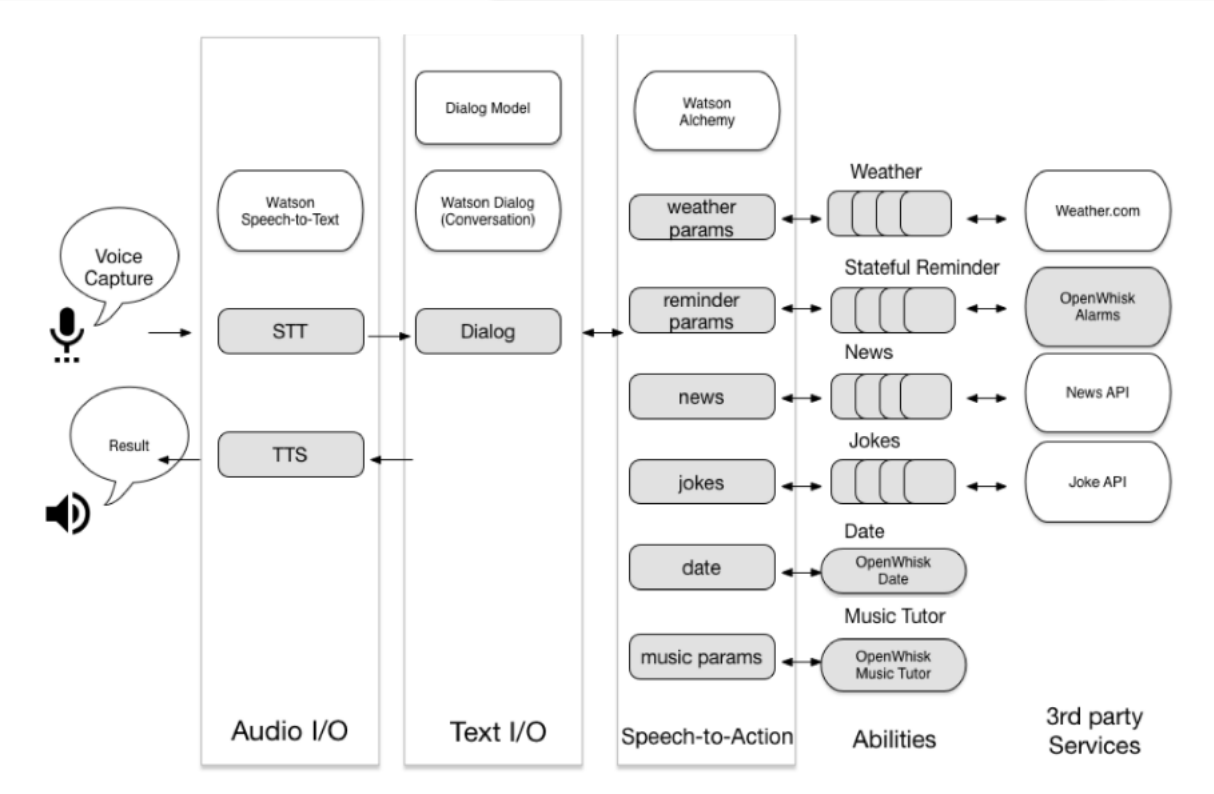
Applications

- Many real-world serverless applications have been proposed in the literature during the past few years.

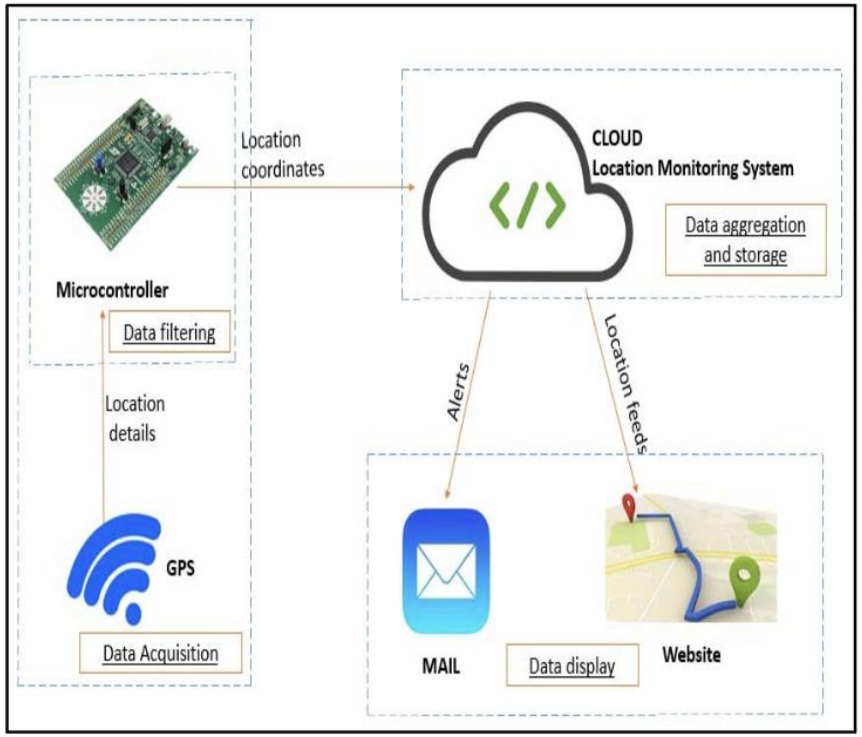
Application	Main Reason	Assessment
Real-time Collaboration and Analytics	Auto-scaling feature	Promising
Urban and Industrial Management Systems	Pricing model	Promising
Scientific Computing	Lower deployment overhead	Fair
Artificial Intelligence and Machine Learning	Pricing model	Fair
Video Processing and Streaming	Lower deployment overhead	Fair
System and Software Security	Auto-scaling feature	Promising
Internet of Things (IoT)	Auto-scaling feature	Promising
E-commerce, Banking and Blockchains	Auto-scaling feature	Fair

Real-time Collaboration and Analytics

- The stateless nature of serverless services makes them an attractive platform for real-time collaboration tools such as instant messaging and chatbots.



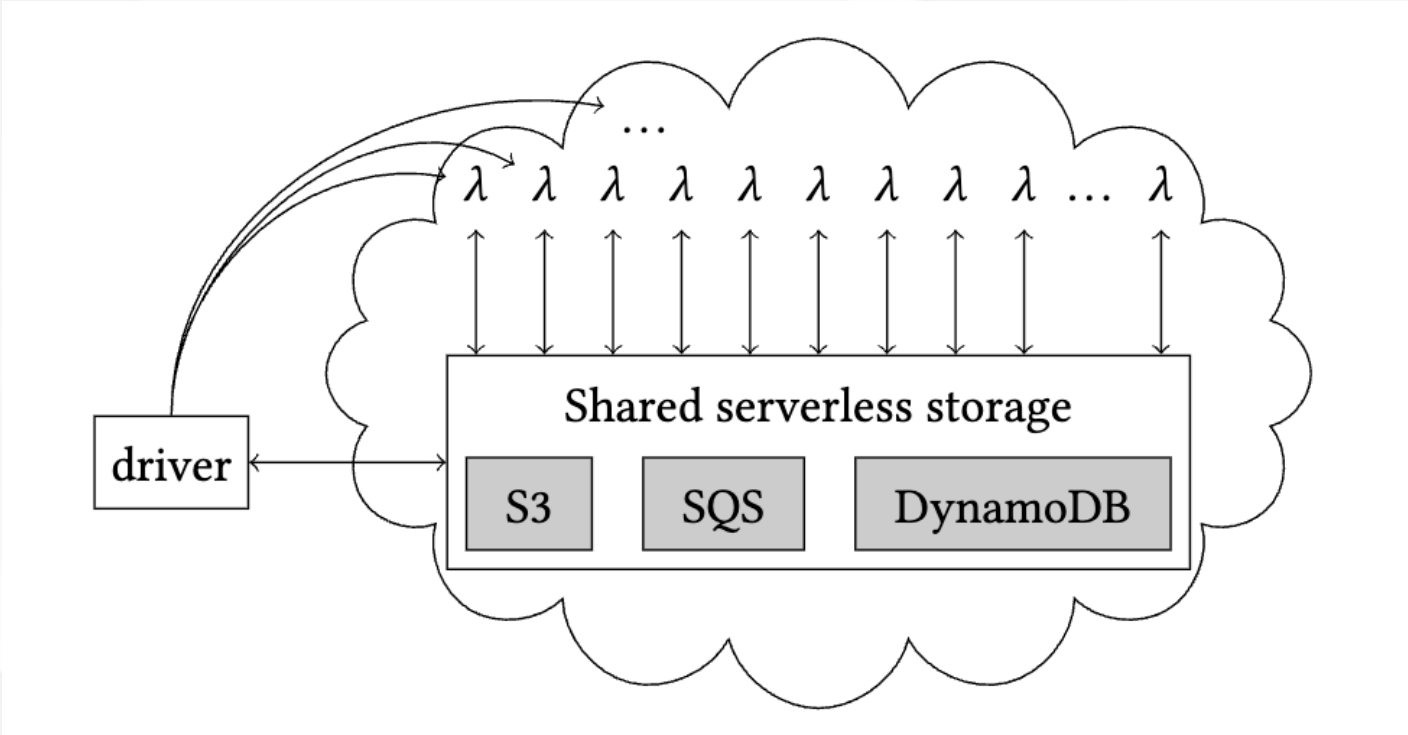
A chatbot based on OpenWhisk



Real-time GPS tracking

Real-time Collaboration and Analytics

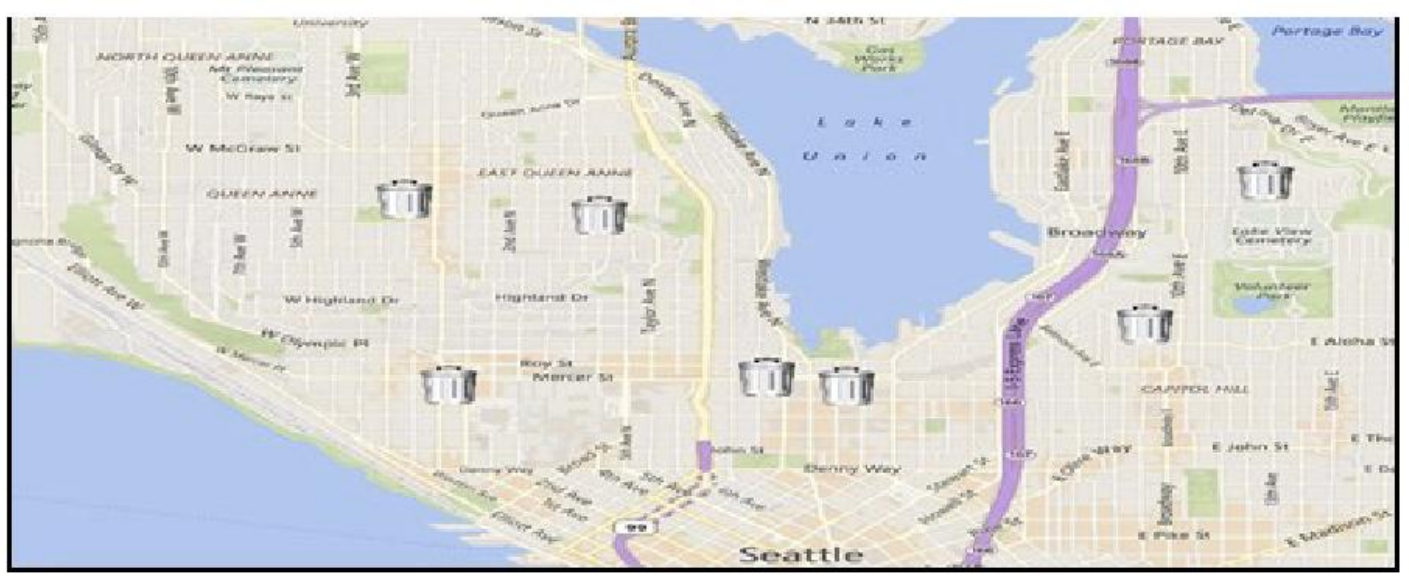
- Serverless services are also utilized for data analytics applications. In these applications, various sources stream real-time data to a serverless service. The service gathers, analyzes, and then represents the data analytics.



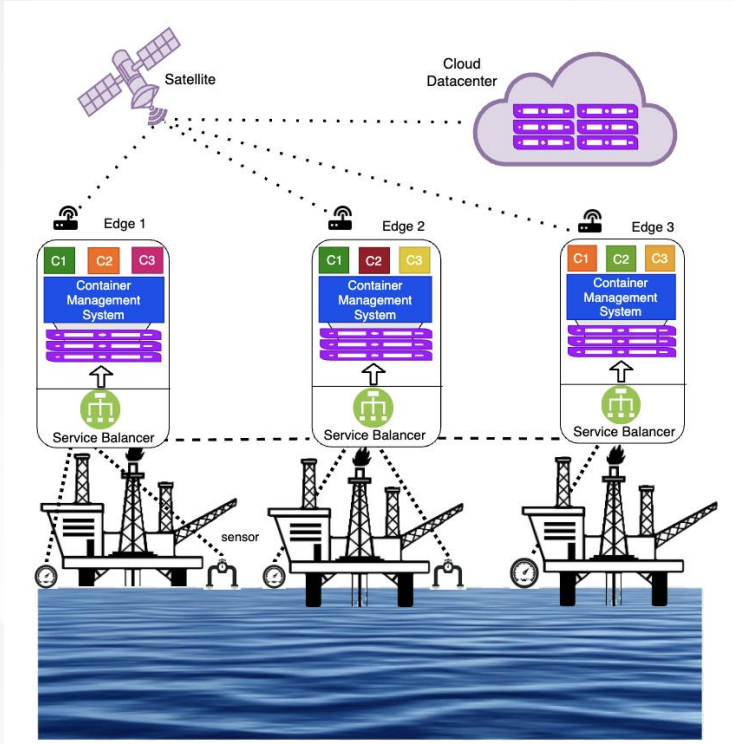
Lambada: a serverless data analytics approach

Urban and Industrial Management Systems

- The pay-as-you-go model of serverless services paved the way for the introduction and implementation of various budget-restricted urban and industrial management systems.

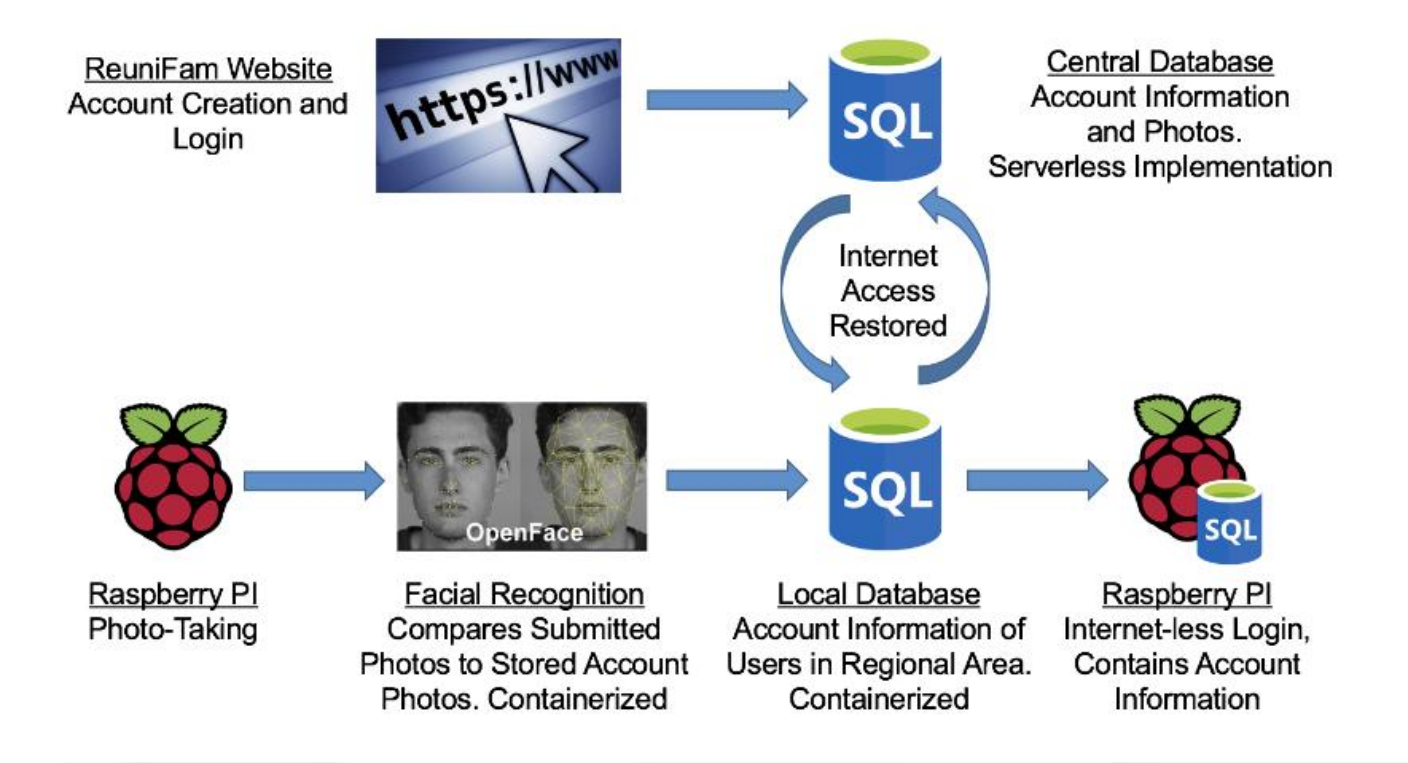


An urban smart waste management system based on Microsoft Azure



An oil and gas field management system

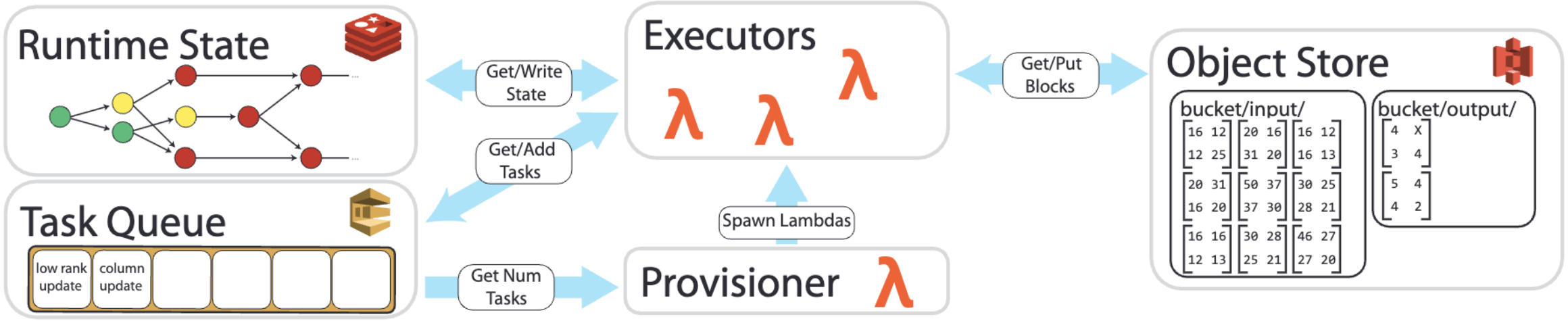
- The distributed nature and auto-scaling feature of serverless services make it an emergency response systems.



System Architecture for the reunification of lost children with their parents after a disaster

Scientific Computing

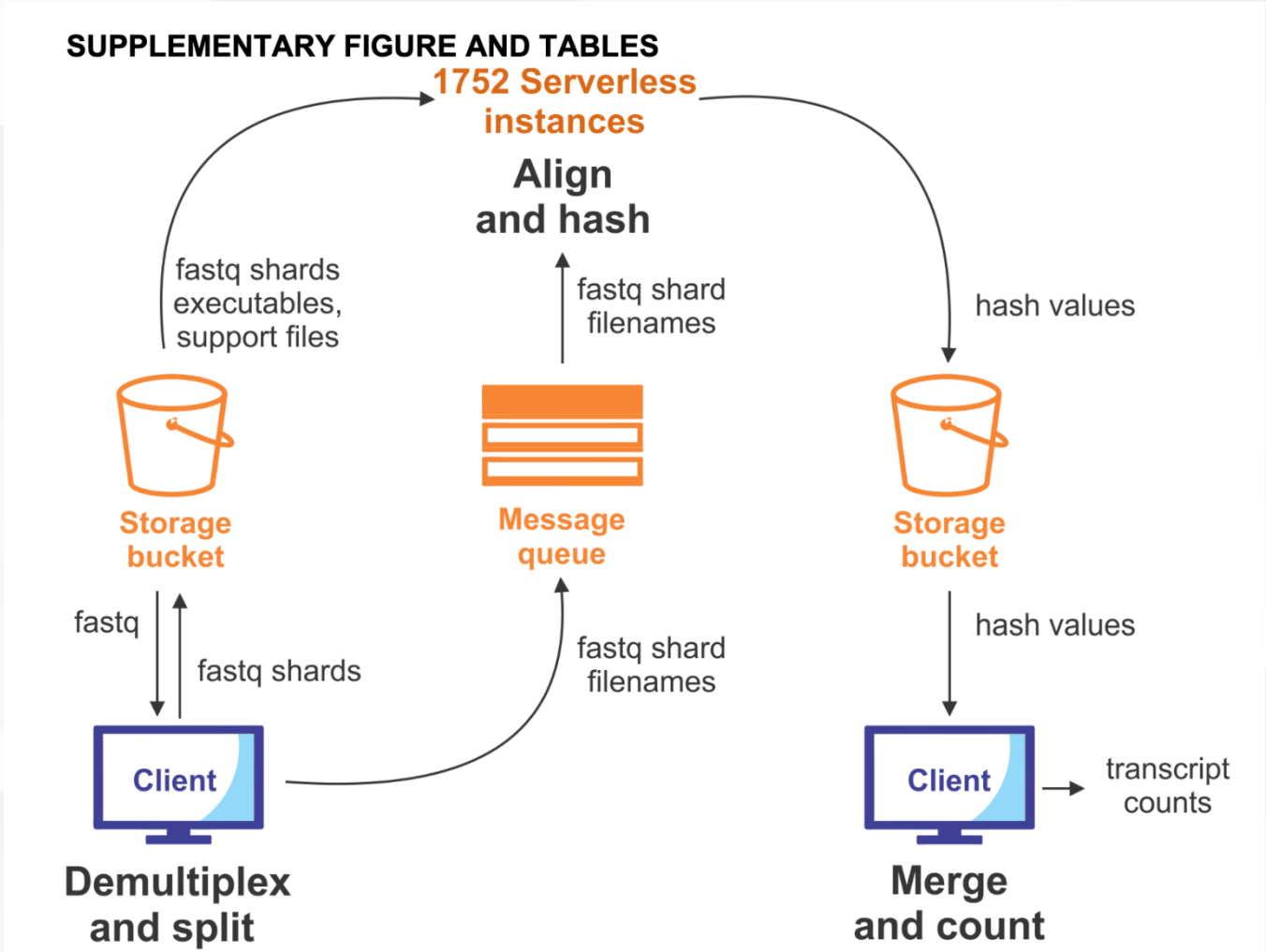
- It has been debated in that serverless computing is not an attractive alternative for scientific computing applications, albeit many studies have focused their attention toward serverless services for those applications.



A serverless tool for linear algebra problems

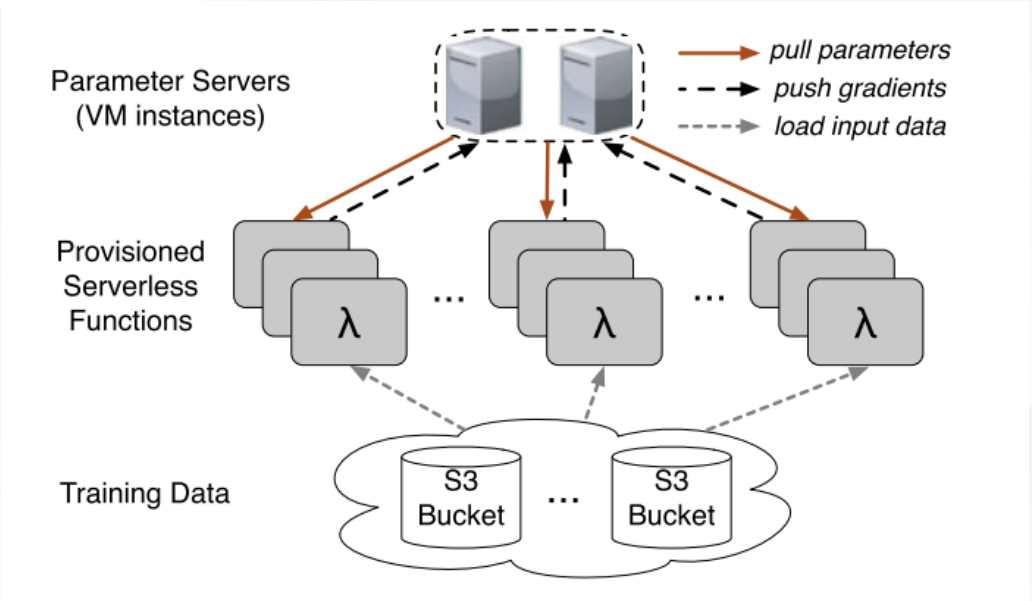
Scientific Computing

- Serverless approaches have been also used in DNA and RNA computing.

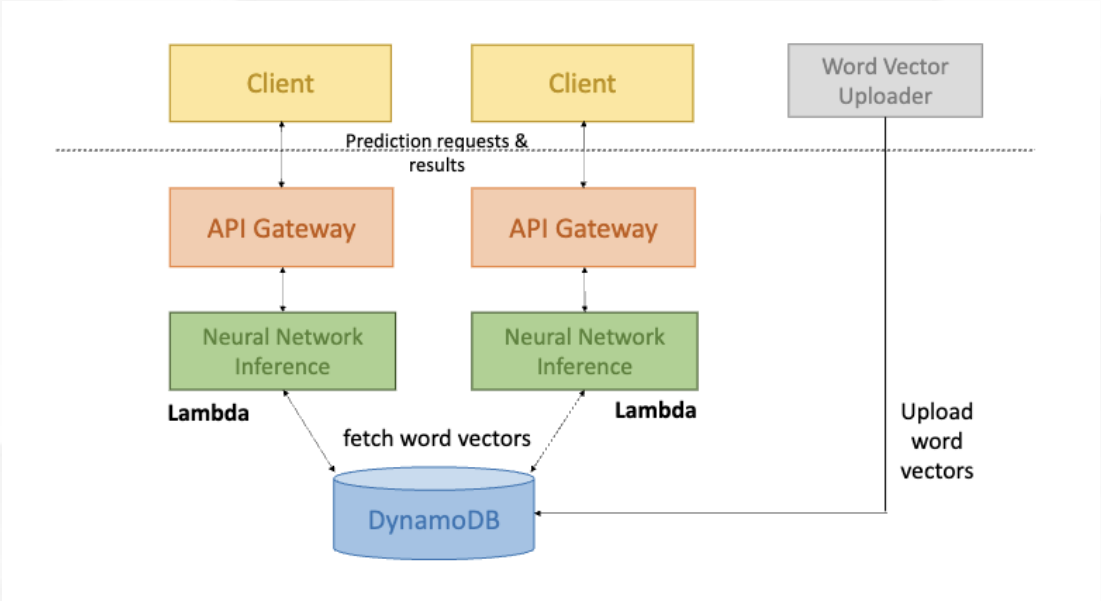


Artificial Intelligence and Machine Learning

- Deep neural networks can benefit from serverless paradigms as they allow users to decompose complex model training into several functions without managing virtual machines or servers.



Serverless model training



Serverless model inference

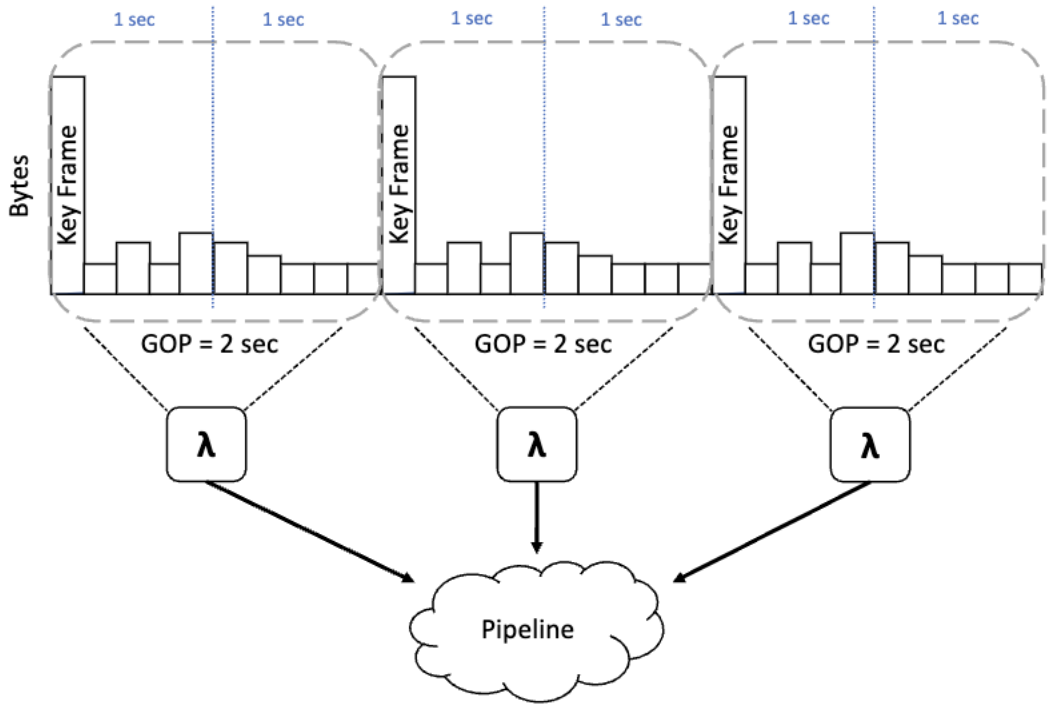
AI and ML: The LLM Era (2024–2026)

Serverless LLM Inference and AI Agents

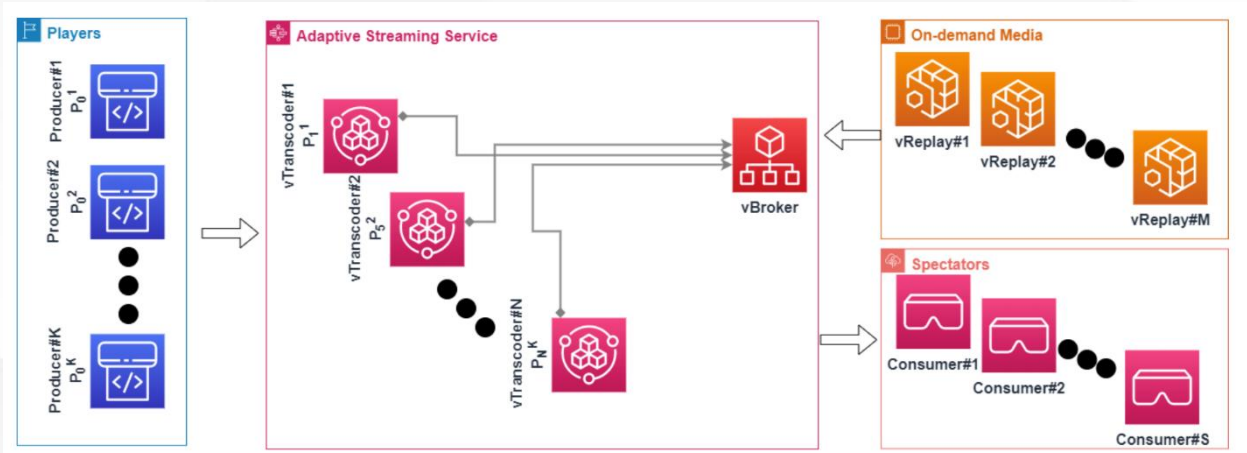
- **Platforms:** Serverless GPUs scale to zero and charge only for use (Google Cloud Run, Alibaba Cloud Function Compute).
- **New bottleneck:** Loading model weights (10–100+ GB) takes far longer than starting a container.
- **Research:** ServerlessLLM (OSDI 2024) loads checkpoints from local memory and SSD; BlitzScale (OSDI 2025, SJTU) multicasts weights over the fast GPU network.
- **Agents:** Long-running, stateful AI agents get one isolated microVM per session (AWS Bedrock AgentCore, 2025).

Video Processing and Streaming

- Serverless approaches have been proposed for video processing.



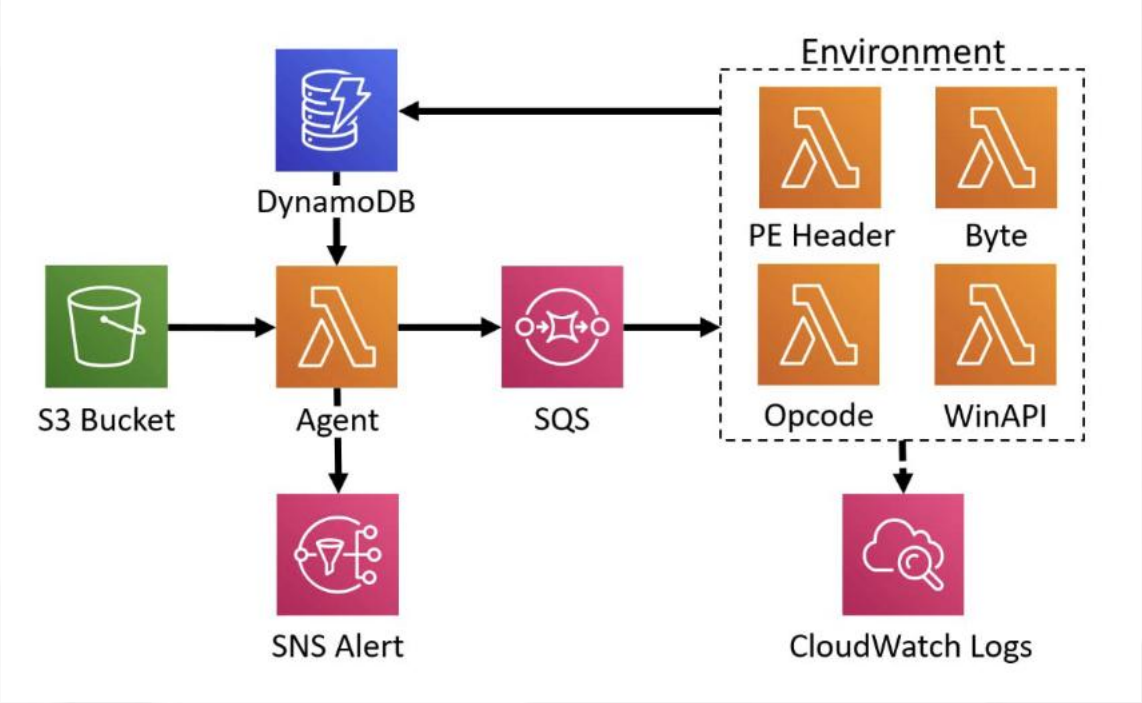
Parallel video processing



A live media streaming based on serverless

System and Software Security

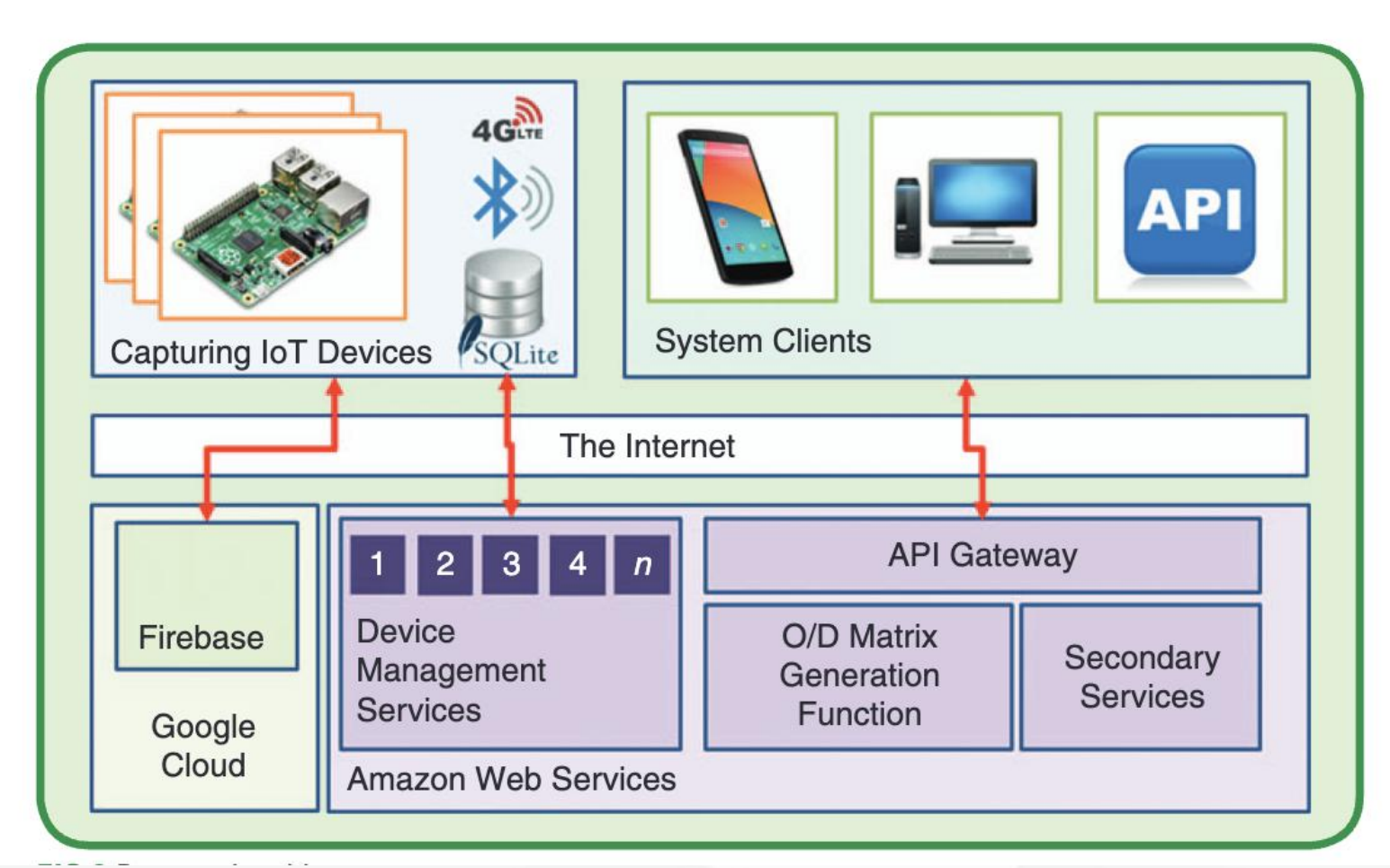
- The power of serverless computing has been leveraged for providing security for various software systems and infrastructures.



Serverless based malware detection

Internet of Things (IoT)

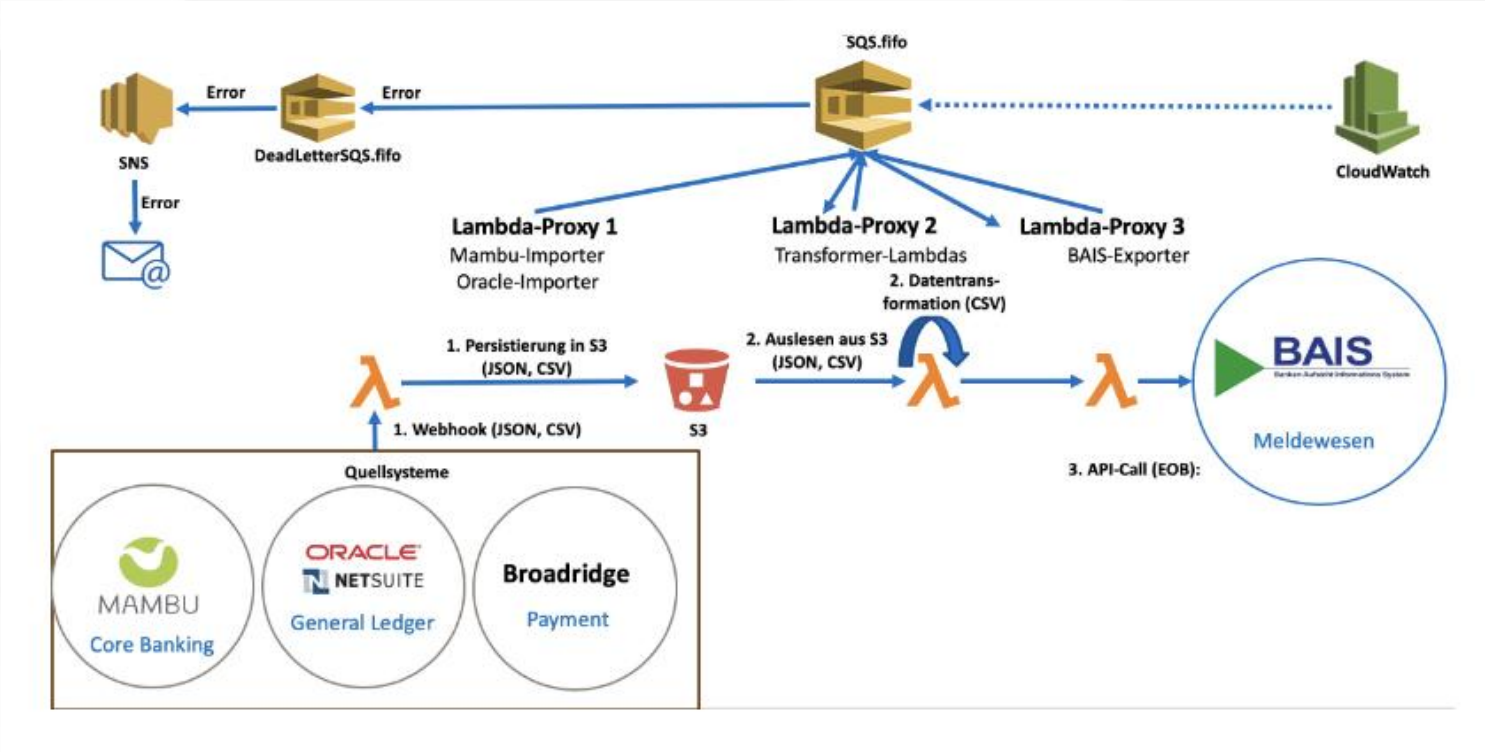
- The serverless computing paradigm has been exploited for various IoT domains.



Serverless based IoT architecture

E-commerce, Banking, and Crypto-currencies

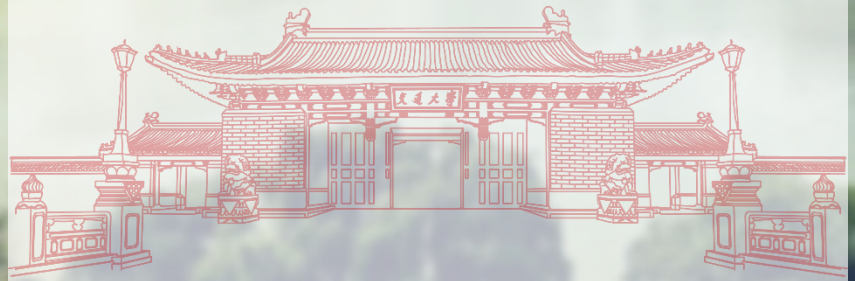
- The inherent scalability of serverless services has enticed few e-commerce and banking applications.



A serverless implementation of a core banking system

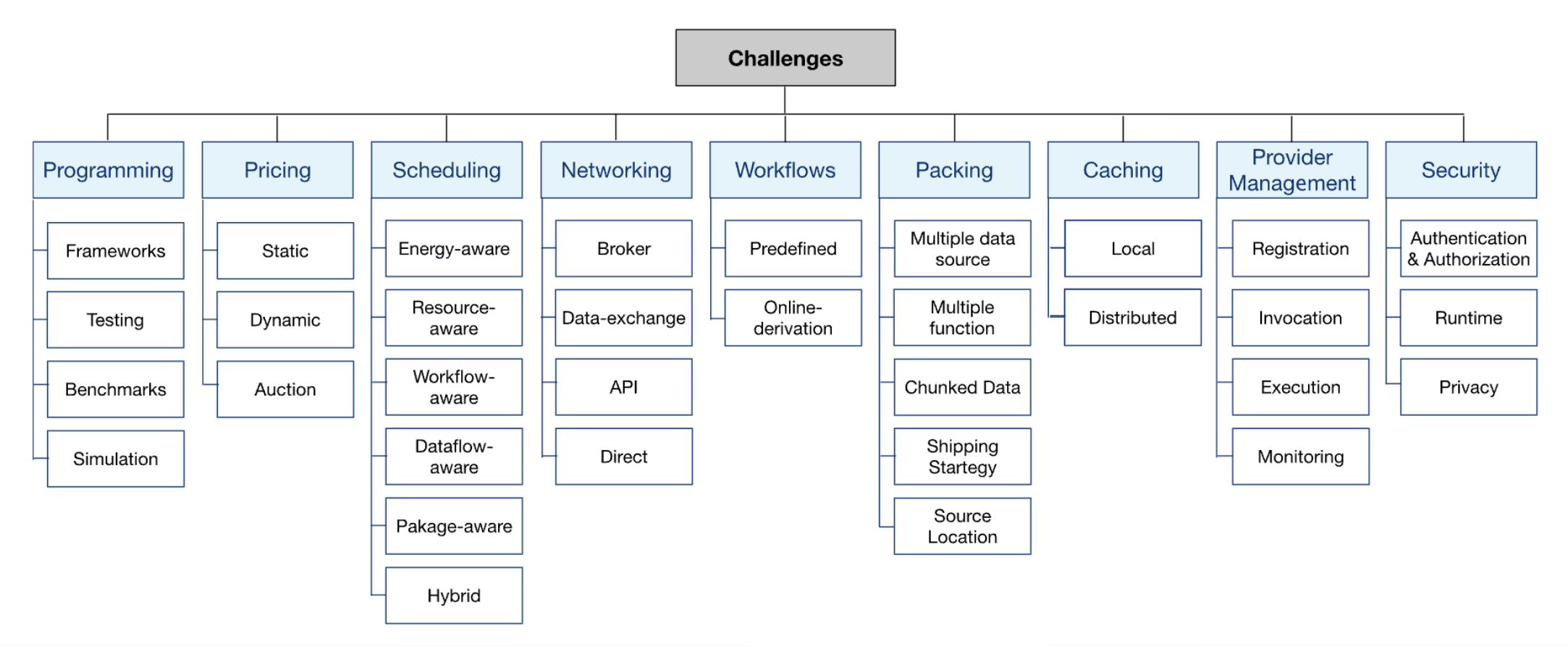
03

Challenges



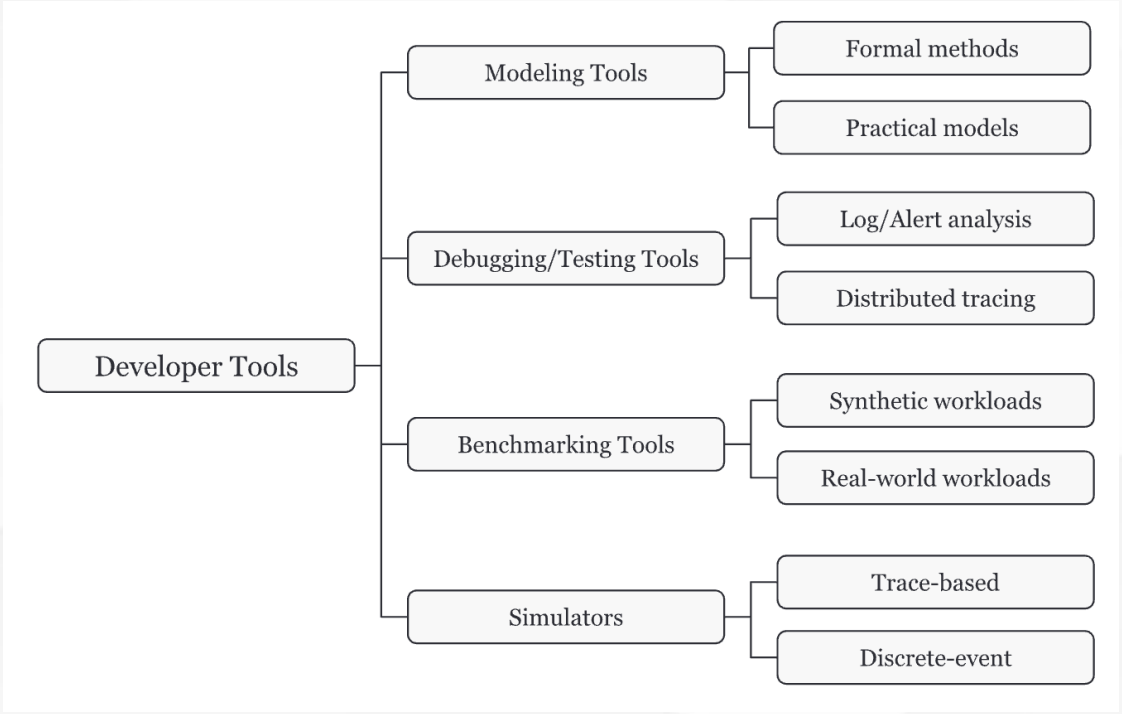
Challenges

- We categorize those challenges into nine topics and survey the existing solutions for each of them.



Developer Tools

- **Challenge Overview:** Moving from traditional, long-running services to small, ephemeral, event-driven functions.
- **Impact:** Requires a different mindset for application design, development, and deployment.
- **Consideration:** Transitioning from monolithic/N-tier to highly distributed micro-functions.



Developer Tools

Debugging & Testing Complexities

- **Issue:** Difficult to debug code that executes briefly and then disappears.
- **Distributed Nature:** Application logic spread across multiple functions and services.
- **Lack of Control:** Limited access to the underlying execution environment makes traditional debugging tools ineffective.

Developer Tools

Immature Tooling & Benchmarking Gaps

- **Tooling:** Lack of mature, integrated development environments (IDEs) specific to serverless.
- **Modeling:** Absence of proper tools to model and analyze serverless application behavior.
- **Benchmarking:** Difficulty in creating realistic benchmarks due to cold starts, variable execution, and dynamic scaling.

Table 3. Comparison of Various Serverless Benchmarking and Experimentation Tools

Reference	Metrics Coverage	Platform(s)	Test Cases/Workloads
DeathStarBench [63]	QoS, tail latency, costs	AWS	Various real-life
ServerlessBench [167]	Communication efficiency, startup latency, and overall performance	AWS, OpenWhisk, and Fn	Real-world workloads
FaaSdom [108]	Latency, cold start, throughput, and cost estimation	AWS, Azure, GCF, and OpenWhisk	Synthetic
PanOpticon [135]	Latency and function chaining	AWS and GCF	Synthetic
vHive [148]	Latency, memory, cold start delay	Knative	FunctionBench [92]

Developer Tools

Research Directions in Serverless Programming

- **Focus:** Developing better IDEs, robust tracing mechanisms for distributed execution.
- **Testing:** New testing frameworks for event-driven, distributed systems.
- **Simulation:** Advanced simulation tools for performance prediction and optimization.

Pricing and Cost Prediction

- **Challenge Overview:** While appealing, the "pay-per-execution" model can lead to unexpected costs in complex serverless applications.
- **Complexity:** Costs accumulate from function invocations, execution duration, memory usage, and data transfer, plus other integrated serverless services.

Table 5. Serverless Cost Prediction Approaches Proposed in the Literature

Reference	Target	Technique	Limitation(s)
Eismann et al. [61]	Google CF	Monte Carlo simulation	Only considers a synthetic workflow and not platforms' workflow services such as AWS Step Functions
Cordingly et al. [54]	AWS, IBM CF	Execution profiling and multiple regression	Focuses on CPU-intensive functions and the effects of external services, and state data stores are not considered
Lin and Khazaei [102]	AWS	PRCP (Greedy algorithm)	State transition costs in workflows, the effects of external services, state data stores, and platform's hardware heterogeneity are neglected

Pricing and Cost Prediction

Diverse Pricing Models

- **Static Pricing:** Fixed rates per resource consumed (e.g., GB-seconds). Most common today.
- **Dynamic Pricing:** Prices fluctuate based on real-time demand or time of day (less common, but a research area).
- **Auction-Based Pricing:** Providers auction off spare capacity, potentially offering lower costs for flexible workloads (academic concept).
- **Recent**: Billing is now per 1 ms (AWS Lambda, 2020); since Aug 2025, the cold-start (INIT) phase is billed too.

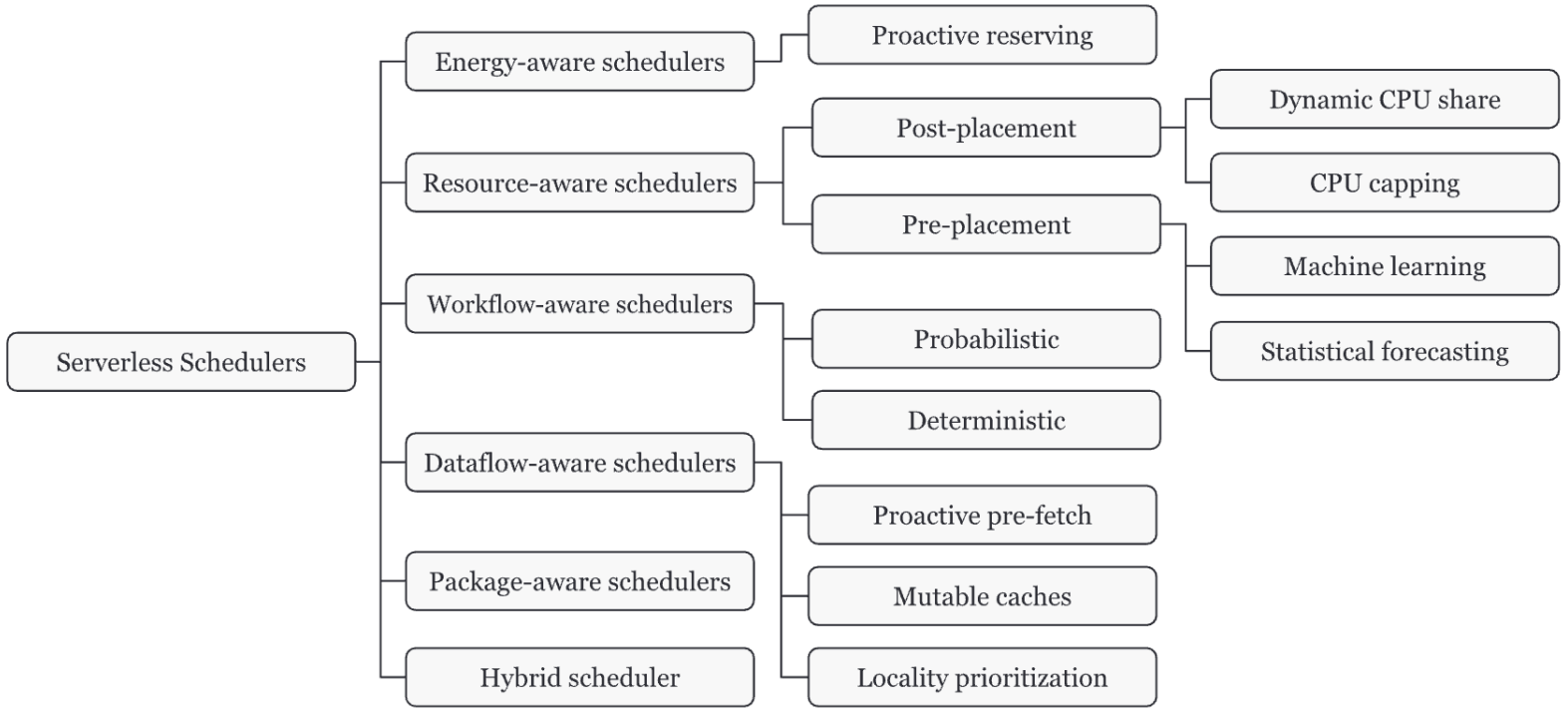
Pricing and Cost Prediction

Cost Optimization & Prediction

- **Difficulty:** Predicting exact costs for highly variable and bursty workloads remains challenging.
- **Goal:** Research aims to develop better cost estimation models and optimization strategies.
- **Strategy:** Finding the "sweet spot" between performance and cost-efficiency.

Scheduling

- **Challenge Overview:** Optimally placing and executing functions across a vast, distributed cloud infrastructure.
- **Goal:** Minimize latency, maximize resource utilization, and manage energy consumption.



Scheduling

Resource & Energy-Aware Scheduling

- **Resource-Aware:** Scheduling functions based on their specific CPU and memory requirements to ensure efficient resource allocation.
- **Energy-Aware:** Placing functions on servers to minimize overall energy consumption (a "green computing" aspect).

Scheduling

Workflow & Dataflow-Aware Scheduling

- **Workflows:** Ensuring correct execution order and dependencies in multi-function workflows.
- **Data Locality:** Scheduling functions close to the data they need to process to reduce network latency.
- **Proactive Fetching:** Fetching data before a function is invoked to minimize delays.

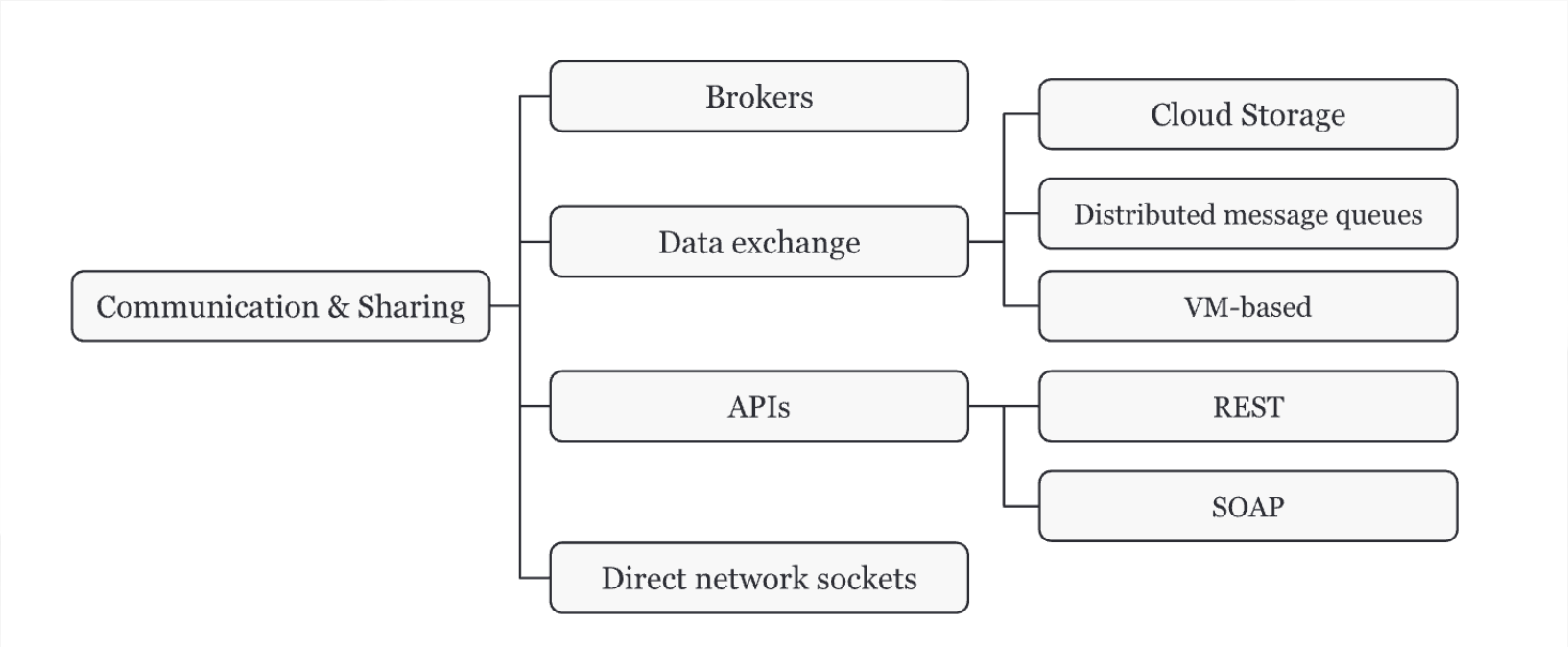
Scheduling

Hybrid & Advanced Scheduling Approaches

- **Hybrid Models:** Combining different scheduling strategies (e.g., energy-aware with data-aware) for complex scenarios.
- **Future Research:** Developing more intelligent, predictive scheduling algorithms that anticipate demand and resource needs.

Networking, Sharing, and Intra-communications

- **Challenge Overview:** Serverless applications often consist of many small, interconnected functions.
- **Impact:** Network latency and overhead from inter-function communication can become a bottleneck, especially in complex workflows.



Networking, Sharing, and Intra-communications

Diverse Communication Patterns

- **Broker-Based:** Functions communicate via message queues or event buses (e.g., Kafka, SQS).
- **Data-Exchange:** Functions interacting through shared object storage (e.g., S3).
- **API-Based:** Functions invoked via API Gateways (HTTP/S calls).
- **Direct Communication:** (Less common/more complex) Functions directly calling other functions.

Networking, Sharing, and Intra-communications

Optimizing Network Paths & Latency

- **Goal:** Minimize network hops, optimize routing, and reduce data transfer times.
- **Research:** Exploring network architectures specifically designed for highly dynamic, bursty serverless workloads.

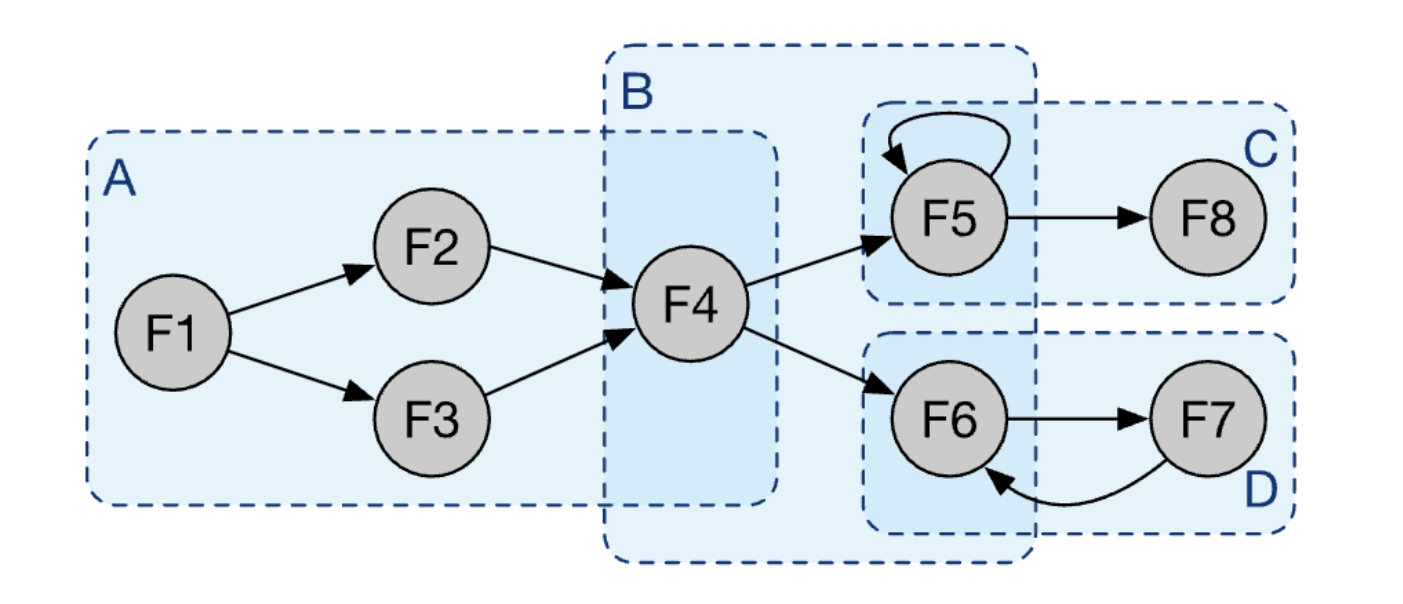
Networking, Sharing, and Intra-communications

The Evolving Landscape of Serverless Networking

- **Focus:** Improving efficiency of data exchange between functions and external services.
- **Edge Computing Synergy:** Leveraging edge locations to reduce latency for geo-distributed applications and IoT.

Serverless Workflows

- **Challenge Overview:** Designing and managing multi-step, dependent tasks using stateless serverless functions.
- **Goal:** Ensure reliability, fault tolerance, and proper state management across a series of function invocations.



Serverless Workflows

Predefined vs. Online-Derived Workflows

- **Predefined:** Workflows with a fixed, known sequence of steps (e.g., using AWS Step Functions, Azure Logic Apps).
- **Online-Derived:** Dynamic workflows where the next step depends on the output or conditions of previous steps (more complex to manage and orchestrate).

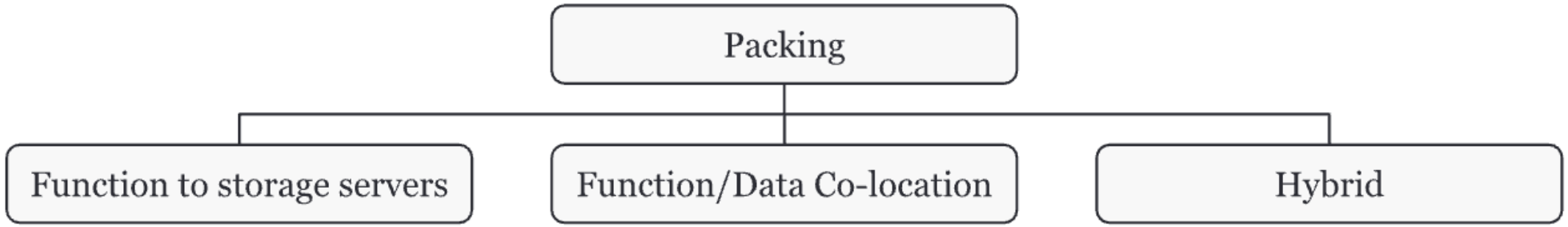
Serverless Workflows

Tools for Workflow Management

- **Importance:** Specialized tools and orchestration engines are essential to define, execute, and monitor serverless workflows.
- **Research:** Improving flexibility, error handling, and performance of workflow orchestrators across different cloud providers.
- **Recent**: Durable execution checkpoints each step and resumes after failures (Temporal; AWS Lambda Durable Functions, 2025).

Packing

- **Challenge Overview:** How to efficiently package function code with its dependencies (libraries, runtimes) for deployment.
- **Impact:** Package size directly affects deployment time and critically, cold start latency.



Packing

Data & Function Co-location (Shipping Strategy)

- **Goal:** Optimize where data resides relative to the functions that process it.
- **"Ship Compute or Ship Data?"**: The fundamental trade-off. Often, moving compute to data is more efficient for large datasets.

Packing

Research in Optimal Packing

- **Focus:** Strategies to minimize package size, optimize cold start times, and improve overall performance through intelligent bundling and deployment.
- **Example:** Only including absolutely necessary libraries, rather than the entire runtime environment.
- **Recent**: Load image blocks on demand (AWS Lambda, ATC 2023) and restore from snapshots (AWS SnapStart; Catalyzer, ASPLOS 2020).

Data Caching

- **Challenge Overview:** Mitigating performance delays (especially cold starts) by effectively caching frequently accessed data or pre-computed results.
- **Importance:** Crucial for achieving low latency and responsiveness in serverless applications.

Data Caching

Local vs. Distributed Caching

- **Local Caching:** Caching data within a specific function instance (ephemeral, not shared across invocations or instances).
- **Distributed Caching:** Using shared, external caching services (e.g., Redis, Memcached) accessible by multiple function instances.
- **Trade-offs:** Performance vs. consistency vs. management complexity.

Data Caching

Intelligent Caching Strategies

- **Focus:** Research into predictive caching (pre-warming caches), adaptive cache invalidation, and integrating caching more seamlessly into serverless platforms.

Provider Management

- **Challenge Overview:** Cloud providers face immense challenges in managing the underlying infrastructure for serverless at hyper-scale.
- **Responsibilities:** Resource allocation, scaling, patching, security, monitoring for millions of concurrent functions.

Provider Management

Isolation & Sandboxing

- **Issue:** Ensuring strong security and performance isolation between different customer functions sharing the same physical hardware.
- **Solutions:** Use of lightweight virtualization technologies like microVMs (e.g., AWS Firecracker) for rapid, isolated execution.
- **Recent**: AI agents that run AI-generated code get one isolated microVM per session (AWS Bedrock AgentCore, 2025).

Provider Management

Monitoring & Tracing at Extreme Scale

- **Challenge:** Collecting, aggregating, and analyzing logs, metrics, and traces from countless short-lived, distributed function invocations.
- **Complexity:** Traditional monitoring tools struggle with the ephemeral and distributed nature of serverless.

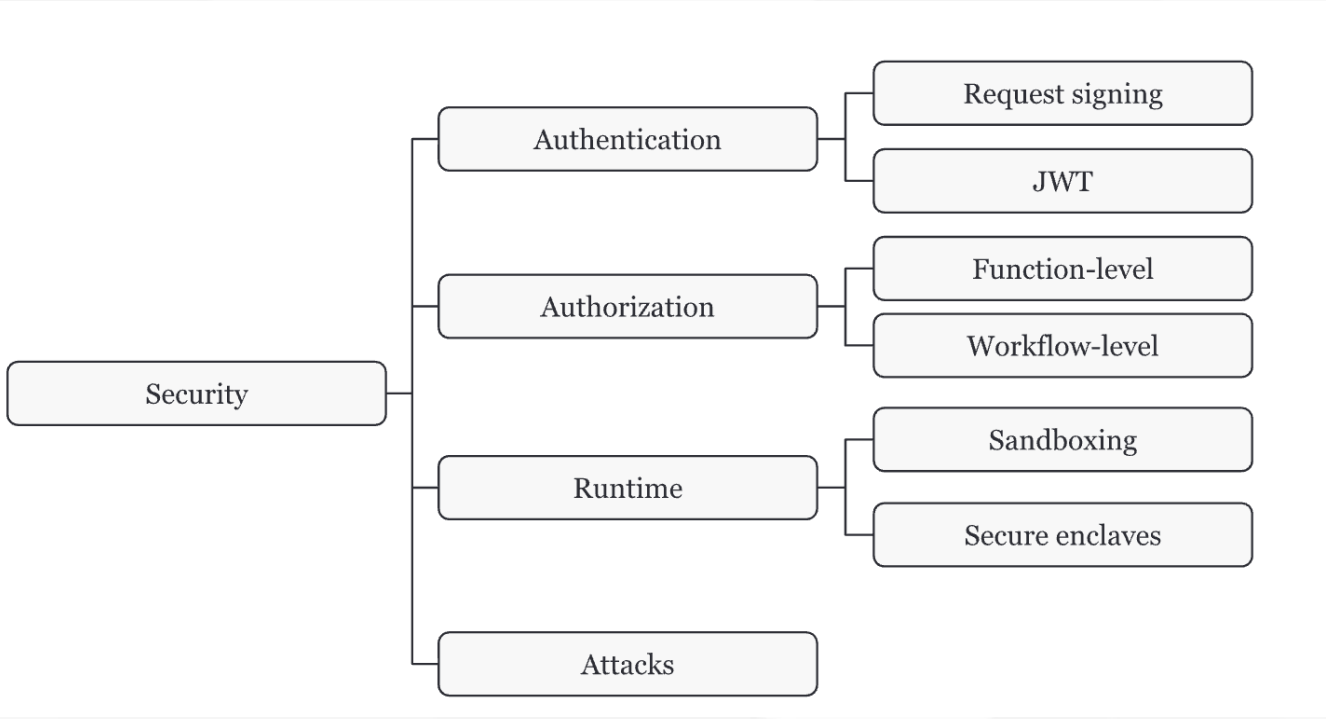
Provider Management

Dealing with Enormous Trace Volumes

- **Problem:** The sheer volume of diagnostic data generated by serverless systems creates its own "big data" challenge for providers.
- **Focus:** Efficient storage, indexing, and analysis of this data to provide actionable insights for debugging and optimization.

Security and Privacy

- **Challenge Overview:** While serverless abstracts away some infrastructure security, it introduces new attack vectors and considerations.
- **Factors:** Shared tenancy, fine-grained permissions, event-driven triggers, potentially larger attack surface from many small functions.



Security and Privacy

Authentication, Authorization & Runtime Security

- **Authentication:** Securing access to and invocation of functions.
- **Authorization:** Implementing least-privilege access for functions and their interactions with other services.
- **Runtime Security:** Protecting against code injection, vulnerable dependencies, and malicious payloads during function execution.

Security and Privacy

Data Privacy & Anonymity

- **Challenge:** Ensuring sensitive data is handled securely throughout the serverless workflow.
- **Privacy:** Preventing inference of user behavior or sensitive information from function invocation patterns, execution times, or locations.

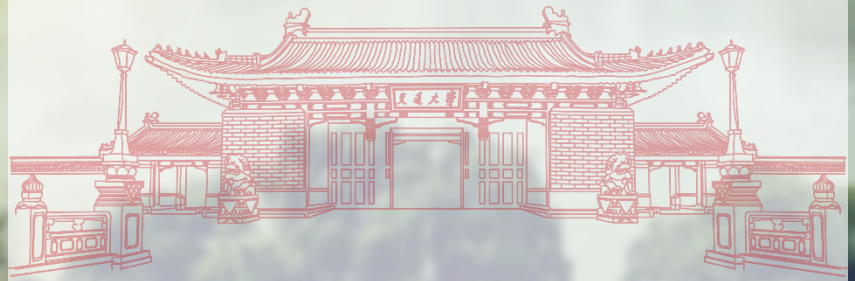
Security and Privacy

Preventing Malicious Serverless Usage

- **Risk:** The ease of deploying and low cost of serverless functions can be exploited by attackers for malicious purposes.
- **Examples:** Launching DDoS attacks, creating botnets, or cryptocurrency mining.
- **Research:** Developing detection and prevention mechanisms for such abuses.

04

Conclusion



Conclusion

- Serverless is powerful, but understanding its complexities is vital for successful adoption.
- Main hurdles discussed: Programming, Pricing, Scheduling, Networking, Workflows, Packing, Caching, Provider Management, and Security.
- Each identified challenge is an active and exciting area of research in both academia and industry.
- These "open problems" drive continuous innovation and lead to more robust serverless solutions.
- New frontiers since the survey: serverless GPUs for LLM inference and runtimes for AI agents



Thanks

饮 水 思 源　爱 国 荣 校